

# Wrinkles in time: Multi-scale patching and super-resolution for efficient time series forecasting

Yuwei Chen<sup>\*</sup> , Wenjing Jia , Qiang Wu

University of Technology Sydney, NSW 2007, Australia

## HIGHLIGHTS

- A hybrid CNN-MLP model is proposed for efficient time series forecasting.
- Multi-scale patching and 2D CNNs enhance seasonal pattern extraction.
- A super-resolution technique refines trend components for better prediction accuracy.
- Competitive and stable performance is achieved while maintaining high computational efficiency and robustness.

## ARTICLE INFO

Communicated by P.N. Suganthan

### Keywords:

Time series  
Forecasting  
CNN  
MLP  
Hybrid models  
Multi-scale  
Patching  
Super resolution

## ABSTRACT

Time series forecasting has become an indispensable part of many fields, from weather prediction to equipment failure forecasting, where accuracy and efficiency directly impact the quality and speed of decision-making. Recently, models with self-attention mechanisms, such as Transformers, have gained widespread attention due to their strong performance in natural language processing and computer vision. However, self-attention mechanisms face certain limitations when applied to time series forecasting tasks, such as high computational complexity and inefficiency in handling long sequences. In contrast, Multilayer Perceptrons (MLPs) exhibit superior efficiency compared to self-attention mechanisms, allowing for faster processing times, especially when dealing with long sequences. However, MLPs have limitations in capturing long-range dependencies and complex patterns within time series data, which can hinder their predictive performance. To this end, we propose a hybrid Convolutional Neural Network–Multilayer Perceptron (CNN–MLP) framework designed to retain the efficiency of MLP-based forecasting while addressing its limitations. Our approach begins by decomposing the input series into its seasonal and trend components. We then apply two targeted enhancements: (i) a multi-scale patching scheme that reshapes the seasonal component into 2D patches processed by a 2D CNN to extract intra- and inter-period patterns; and (ii) a diffusion-based super-resolution module that enriches the trend component with fine-scale temporal detail. Collectively, these innovations introduce wrinkles in time, improving feature extraction at both global and local scales and yielding a model that achieves competitive accuracy while maintaining low parameter counts, low inference latency, and stable performance across diverse forecasting scenarios. Our source code is available at <https://github.com/sonnybjs/Wrinkles-in-time>.

## 1. Introduction

In recent years, the self-attention mechanism has gained widespread use in tasks such as natural language processing, image recognition, and time series forecasting due to its outstanding performance in capturing global features. Particularly in long-sequence forecasting, works derived from the Transformer, such as Autoformer [1], Informer [2], and iTransformer [3] have enhanced predictive results by modelling

long-range dependencies and improving the model's ability to capture global patterns within sequences. However, despite its strong representational capabilities, the computational complexity of the self-attention mechanism, which grows quadratically with the input size, remains a significant issue, especially when dealing with large-scale data. To address this challenge, the work from Guo [4] proposes using a two-layer linear network as a replacement for the self-attention mechanism, aiming to reduce computational complexity while retaining the ability to

<sup>\*</sup> Corresponding author.

Email addresses: [Yuwei.Chen@student.uts.edu.au](mailto:Yuwei.Chen@student.uts.edu.au) (Y. Chen), [Wenjing.Jia@uts.edu.au](mailto:Wenjing.Jia@uts.edu.au) (W. Jia), [Qiang.Wu@uts.edu.au](mailto:Qiang.Wu@uts.edu.au) (Q. Wu).

<https://doi.org/10.1016/j.neucom.2026.133021>

Received 6 March 2025; Received in revised form 2 January 2026; Accepted 10 February 2026

Available online 14 February 2026

0925-2312/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

model global features. Although several high-impact studies [5–7] have demonstrated the effectiveness of self-attention, its computational cost remains a key bottleneck limiting its broader application. This limitation is particularly critical in latency-sensitive forecasting settings, where predictions must be produced within tight time budgets for operational decision-making (e.g., large-scale monitoring and control systems such as network traffic forecasting in cellular base-station operations). Therefore, inference efficiency and memory footprint are treated as first-order constraints in addition to forecasting accuracy.

In contrast, Multi-Layer Perceptrons (MLPs) exhibit superior computational efficiency, with significantly lower complexity compared to the self-attention mechanism. However, MLPs have notable limitations in capturing both local and global features, making it difficult to effectively model complex dependencies within input data [8]. Research such as [9] explores methods to improve MLP performance by reducing the computational load in sparse contexts. However, using MLPs as a replacement for self-attention mechanisms in time series forecasting has not been a popular approach since both methods have distinct points of contention.

However, one of the latest state-of-the-art works, TimeMixer [10], demonstrates that purely MLP-based architectures, equipped with multi-scale mixing mechanisms, can achieve remarkable performance without relying on self-attention. This model leverages multi-scale sequence information to capture global patterns effectively. Nevertheless, for data with strong seasonal fluctuations or intricate local patterns, TimeMixer — like many MLP-only models — may struggle to capture localised periodicities. It also does not explicitly address the separate modelling of the strong trend component, potentially limiting its performance on datasets with significant long-term trends. These observations motivate a decomposed architecture where seasonal regularities are handled by 2D CNN patching and long-range drifts are refined by trend super-resolution, rather than relying on a single dense mixing pathway.

To tackle these gaps, we propose a novel hybrid CNN-MLP framework that decomposes time series into seasonal and trend branches. By using 2D patching plus CNNs for the seasonal branch, our model robustly extracts repetitive patterns at multiple frequencies, while a super-resolution module refines the trend branch to expose finer-grained variations. Through this decomposition-based strategy, the proposed approach achieves high forecasting accuracy on a diverse set of real-world benchmarks, striking a favourable balance between predictive precision and computational efficiency. We therefore adopt multi-scale patching to reduce token count and expose periodic structure before prediction, while keeping the predictor lightweight.

While prior MLP mixers improve efficiency, they typically treat seasonal and trend dynamics within a single pathway. Our approach explicitly separates these factors—2D CNNs for seasonal structure and super-resolution (SR) for trend refinement—before a lightweight MLP readout. In particular, this work retains the computational efficiency and high performance of MLPs while incorporating insights from the TimesNet framework [11], particularly its use of 2D CNNs to capture seasonal patterns. After combining CNNs and patching techniques, our approach decomposes time series data, transforms it into 2D representations, and selectively enhances the trend component. By leveraging multi-scale techniques and 2D kernel transformations, the model improves feature capture efficiency. In this way, the proposed model not only bridges the performance gap between CNNs and MLPs in time series forecasting but also meets the growing demand for a balance between predictive accuracy and computational feasibility.

Our contributions are summarised as follows:

- We propose a hybrid model combining CNNs and MLPs for time series forecasting, leveraging multi-scale patching to enhance feature extraction at both global and local levels.
- We introduce an efficient fusion method for seasonal and trend components, improving accuracy by handling cyclic and long-term patterns separately.

- Our model outperforms state-of-the-art self-attention-based approaches, achieving superior performance and efficiency on multiple forecasting benchmarks.

## 2. Related work

### 2.1. Self-attention mechanisms in time series forecasting

Early time series models primarily used recurrent neural networks (RNNs) [12], long short-term memory networks (LSTMs) [13], or one-dimensional convolutional networks (1D CNNs) [14]. These models can model sequence dependencies to a certain extent, but they also have significant limitations. For instance, as pointed out in [15], RNNs and LSTMs often encounter issues with vanishing or exploding gradients when processing long sequences, making it difficult to capture long-range dependencies. Meanwhile, CNNs focus more on local features and lack a global perspective. In this context, the self-attention mechanism allowed each time point to attend to the entire sequence, thereby building relationships between all positions in a single computation and significantly enhancing the ability to capture global dependencies.

The self-attention mechanism was initially introduced in [16]. It achieved great success in the field of natural language processing and subsequently expanded rapidly into other domains, such as image recognition and time series forecasting. In the field of time series forecasting, Transformer-based derivative models such as Autoformer [1] and Informer [2] have been customised and optimised for the specific characteristics of time series data, achieving excellent results and quickly becoming mainstream within the time series analysis community. Autoformer [1] uses a decomposition module to split time series data into trend and seasonal components, modelling each separately to better capture periodicity and long-term dependencies. Informer [2] employs a sparse self-attention mechanism that reduces unnecessary computations, improving model efficiency and addressing the computational bottleneck of long-sequence processing. FEDformer [6] further extends the sparse attention mechanism, striking a better balance between computational complexity and performance. More recently, iTransformer [3] inverts the modelling dimensions so that attention operates over variate tokens rather than temporal tokens, improving multivariate dependency learning without altering core components of the Transformer; DeformableTST [17] replaces full attention with a deformable/sparse attention that focuses on salient timesteps and reduces reliance on heavy patching.

Despite the excellent performance of self-attention mechanisms in time series forecasting tasks, their computational complexity is a major limiting factor. Standard self-attention operations require calculating similarity for each pair of positions in the input sequence, causing computational complexity to grow quadratically with sequence length. For long-sequence data with high sampling rates (such as traffic flow or weather data), this computational cost is often prohibitive. To address this issue, researchers have proposed various alternative methods or optimisation strategies. Models such as Linformer [18], Longformer [19], and Performer [20] attempt to reduce the computational load by sparsifying the attention matrix or introducing low-rank approximations while still preserving the global modelling capability of self-attention. In parallel, GRAformer [21] streamlines the Transformer stack by introducing a gated residual attention unit as an efficient alternative to a heavy feed-forward block, and TF4TF [22] explicitly models multi-semantic information in the time-frequency domain within a Transformer, further improving long-horizon performance under modest compute.

Furthermore, Zeng et al. [23] pointed out that, compared to traditional time series models or other lightweight architectures, Transformers may not offer significant advantages for shorter sequences or low-frequency data. This finding encourages researchers to reconsider the application boundaries of Transformers and explore more efficient alternatives that reduce computational costs while preserving the ability to capture global features. These insights provide a theoretical basis for

optimising self-attention mechanisms or adopting other architectures, such as hybrid models that combine MLP and CNN. Consistent with these observations, our method externalises seasonal–trend handling and leverages lightweight modules (2D seasonal patching + trend refinement) as the upstream of the predictor, which complements the above Transformer advances by reducing reliance on full attention for long-range structure. Beyond attention-centric designs that incorporate decomposition as an internal block (e.g., Autoformer [1] and FEDformer [6]), recent studies revisit decomposition as an explicit inductive bias and combine it with multi-scale processing in lightweight hybrids. Hu et al. [24] proposed an Adaptive Multi-Scale Decomposition framework that learns scale-aware component separation, demonstrating that adaptive decomposition can be beneficial when characteristic periods vary across datasets and regimes. From a complementary perspective, Cao et al. [25] presented MSPatch, where multi-scale patch mixing implicitly captures component-wise dynamics through cross-scale interactions, illustrating that decomposition effects can emerge from multi-resolution mixing rather than a standalone split. In industrial forecasting settings, Oikonomou et al. [26] further demonstrated that combining decomposition-processed streams with patching/predictor pipelines via late fusion can improve robustness under heterogeneous operating conditions. These works collectively motivate positioning the proposed moving-average seasonal–trend split as a lightweight but explicit front-end decomposition that aligns with recent efficient hybrid and multi-scale directions.

## 2.2. MLP-based approaches for time series data

Despite the strong performance of self-attention mechanisms in the field of time series, their high computational cost remains a bottleneck that urgently needs to be addressed. Consequently, in recent years, some researchers have begun exploring alternative architectures to achieve efficient time series feature extraction without relying on self-attention mechanisms. This includes using multi-layer perceptrons (MLPs) for time series forecasting. MLPs, composed of fully connected layers, offer a computational advantage over self-attention mechanisms [9], providing an efficient solution for large-scale data processing with stringent latency requirements. Many tasks that involve processing large amounts of time series data have adopted MLP architectures, as seen in works such as [27,28].

In addition to early MLP applications, recent work utilising MLP architectures has also achieved excellent results. The latest state-of-the-art model, TimeMixer [10], introduces a decomposable multi-scale mixing mechanism that significantly reduces computational complexity and training time while maintaining high-accuracy predictions. Experimental results show that TimeMixer outperforms the current state-of-the-art Transformer models across multiple metrics, including prediction accuracy, computational efficiency, model interpretability, generalisation ability, and long-range forecasting. Although MLPs are computationally efficient and well-suited for large-scale data processing, they still lag behind models such as self-attention mechanisms and CNNs in terms of fine-grained feature extraction and sequence relationship modelling. Very recent studies report competitive lightweight MLP variants: FTMLP [29] adopts a Feature–Temporal block to jointly leverage channel and temporal cues, while TCM [30] employs an affine-transformation-based lightweight head to stabilise training and improve efficiency under standardised long-term time series forecasting (LTSF) settings. These results further support the view that MLP backbones can remain highly competitive when paired with the right inductive biases. Our design follows this efficiency thread but addresses a common limitation of single-path mixers by decoupling seasonal and trend signals upstream of the MLP.

Subsequent works [27,31] utilising frequency-domain MLPs have further enhanced the application of MLPs in the time series field. Frequency-domain MLPs transform input data from the time domain to the frequency domain, capturing the frequency characteristics of the

data and performing learning and prediction in the frequency domain. Most recently, WPMixer [32] extends the MLP-mixer family by incorporating multi-level wavelet decomposition and patch mixing to jointly capture features both in time and frequency domains. By coupling wavelet-based multi-resolution analysis with patching, WPMixer achieves state-of-the-art long-term forecasting accuracy while remaining highly efficient. These approaches allow for a more effective capture of the periodicity and trend in time series data, thereby improving prediction accuracy. The above lightweight MLP advances mitigate part of this gap; our hybrid addresses the remaining issues by injecting CNN-extracted seasonal structure and diffusion-style trend refinement upstream of the MLP. In our design, we follow this efficiency thread but explicitly externalise seasonal–trend structure, where a 2D seasonal patch branch provides localised periodic features, and a super-resolved trend branch refines coarse trends before the final MLP readout, so the MLP can focus on residual interactions without carrying the full burden of long-range structure.

## 2.3. CNNs in time series forecasting

CNN also plays critical roles in the field of time series analysis due to its powerful local dependency and pattern-capturing abilities. Unlike early models such as RNNs or LSTMs that process data sequentially, CNNs use convolutional layers to identify spatial patterns across time steps, efficiently modelling short-term dependencies. This characteristic has led to the widespread application of CNNs in tasks involving large-scale data or high-dimensional inputs, such as financial forecasting [33], sensor data analysis [34], and traffic prediction [35].

With the development of the time series forecasting field, the application of CNNs has also expanded. For example, TimesNet [11] proposes a CNN-based method for time series analysis, but unlike traditional 1D CNNs, it transforms time series data into a 2D space and uses 2D convolutional kernels to capture complex variation patterns in the series. TimesNet combines the powerful representational capabilities of CNNs with the ability to capture periodicity and variation patterns in time series analysis, making it an innovative framework for time series analysis. More recently, Times2D [36] generalises this 2D view by introducing a multi-period decomposition block and derivative-based maps before 2D convolution, achieving high accuracy on both short- and long-horizon settings. Our seasonal branch follows the same 2D manner but relies on trajectory (phase–cycle) unfolding with Inception-style kernels, and pairs it with a trend super-resolution path—components typically absent in CNN-only designs. In this paper, “trajectory (phase–cycle) unfolding” simply means reshaping a 1D series into a 2D layout where each column is a within-period “phase” (e.g., hour of day) and each row is a successive “cycle” (e.g., different days). This layout is a classic trick in time-series analysis—often called a trajectory or Hankel view in Singular Spectrum Analysis—and is widely used to make seasonal structure explicit so that 2D filters can act across phases and across cycles. In the same spirit as prior 2D time-series works such as TimesNet [11] and the multi-period layout in Times2D [36], we also adopt this trajectory layout to build our 2D seasonal branch.

## 2.4. Multi-scale and patching techniques

In the field of multivariate time series analysis, in addition to the common innovations focused on model architecture, such as Autoformer [1] and Informer [2], there is another category of work that emphasises innovations in data input and preprocessing, represented by approaches such as PatchTST [37] and HdMixer [38].

The application of patching has been widely adopted in the field of computer vision. Notable models such as Vision Transformers (ViT) [39], Pyramid Vision Transformer [40], and Swin Transformers [41] utilise patching for data preprocessing, where images are segmented into patches and processed as tokens before being fed into the encoder. These works have demonstrated the effectiveness of patching in computer vision tasks. Inspired by the success of patching

in CV, this technique has also seen widespread adoption in time series analysis in recent years.

Taking PatchTST [37] as an example, PatchTST incorporates two key design elements, patching and channel independence, that enable the extraction of local semantic information while leveraging a longer historical context for forecasting. It has demonstrated outstanding performance across various tasks, including supervised learning, self-supervised representation learning, and transfer learning. Through the patching operation, the number of input tokens can be significantly reduced thereby significantly lowering computational time and memory complexity.

At the same time, patching allows for an extended input window without increasing the number of input tokens, further improving prediction accuracy. Both of these advantages have led to superior performance in self-supervised pretraining and transfer learning tasks [42,43], indicating that the learned representations possess strong generalisation capabilities.

In PatchTST and a series of subsequent Transformer-based works, the combination of patching for time series data processing and Transformer model architecture has gradually become the mainstream approach for time series forecasting. However, many of the earlier works utilised a fixed time window for patch processing, limiting the model's ability to capture patterns across different scales. This has given rise to a new research direction: how to design multi-granularity patching methods to enhance the modelling capability of the patch-based forecasting architectures.

Recent studies further strengthen this direction by explicitly designing multi-scale patching and scale-adaptive pathways. TimeSQL [44] introduces multi-scale 2D patching together with a smooth quadratic loss to improve robustness under noise and outliers, demonstrating that patch granularity is a strong lever for multivariate forecasting. Pathformer [45] proposes a multi-scale Transformer with adaptive pathways that dynamically selects and aggregates scale-specific representations, improving generalisation across diverse temporal dynamics. These works corroborate the effectiveness of multi-scale patching; in contrast, the proposed framework couples multi-scale patching with an explicit seasonal-trend decomposition and a lightweight CNN-MLP backbone, while reserving the trend branch for super-resolution refinement. More broadly, recent hybrid forecasters increasingly couple multi-scale tokenisation (e.g., patching/mixing) with explicit or implicit decomposition to better separate recurring and slowly-varying dynamics. The adaptive decomposition view in AMD [24] emphasises learning scale-specific component separation, whereas MSPatch [25] achieves a similar goal implicitly through multi-scale patch mixing. In parallel, late-fusion frameworks [26] suggest that maintaining separate processing streams (e.g., decomposition-informed representations and patch-based predictors) and fusing them at a later stage can improve stability in real industrial environments. Compared with these directions, the proposed framework adopts a deliberately simple moving-average split as an explicit decomposition front-end, uses multi-scale 2D patching primarily to strengthen seasonal representation learning, and refines the trend component via a dedicated super-resolution pathway. This design aims to preserve the efficiency benefits of patching while making the seasonal-trend separation explicit and controllable.

From an implementation perspective, patch construction is governed by the patch length (window size) and the sliding stride, which together determine the temporal granularity and the overlap pattern between neighbouring patches. Multi-granularity designs therefore instantiate multiple configurations to expose complementary scale information, improving robustness when characteristic periods are unknown or mixed. Building on this idea, representative examples include MSD-Mixer [46] and MTST [47], which introduce multi-scale patch-based Transformer models that achieve strong forecasting performance across multiple resolutions.

Beyond Transformer backbones, patching has also been integrated with CNNs, further demonstrating its general effectiveness. For example, HDMixer [38] combines patching with CNN-based mixing blocks and reports state-of-the-art performance, outperforming several Transformer-based baselines.

### 3. The proposed method

This work tackles the challenge of enhancing MLPs for time series forecasting, enabling the effective capture of both global and local features without relying on computationally intensive self-attention mechanisms. By improving MLPs' ability to model complex dependencies, our approach aims to achieve a balance between computational efficiency and strong feature extraction, providing a scalable solution for accurate and efficient time series prediction.

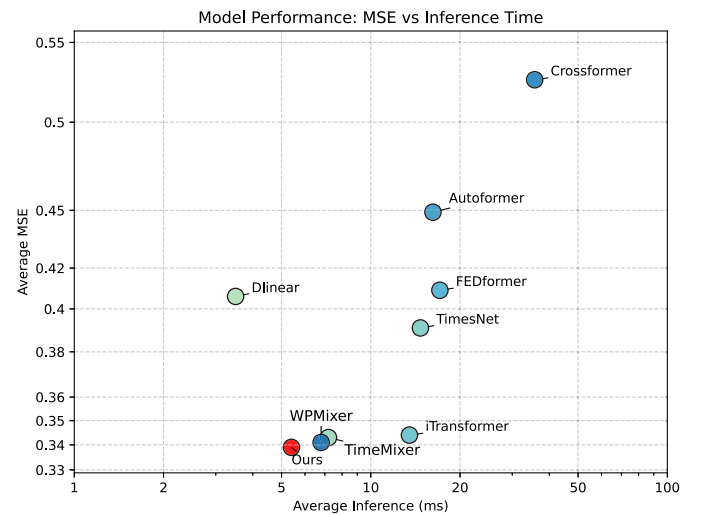
#### 3.1. Problem formulation and input-output pipeline

Let  $\mathbf{Z} \in \mathbb{R}^{T \times C}$  denote a multivariate time series with  $C$  variables. Given a look-back window length  $L$  and forecasting horizon  $H$ , the task is to learn a mapping from the input window  $\mathbf{X} = \mathbf{Z}_{t-L+1:t} \in \mathbb{R}^{L \times C}$  to the future target  $\mathbf{Y}^{gt} = \mathbf{Z}_{t+1:t+H} \in \mathbb{R}^{H \times C}$ , producing the prediction  $\mathbf{Y} \in \mathbb{R}^{H \times C}$ .

The proposed model consists of two branches connected by explicit decomposition:

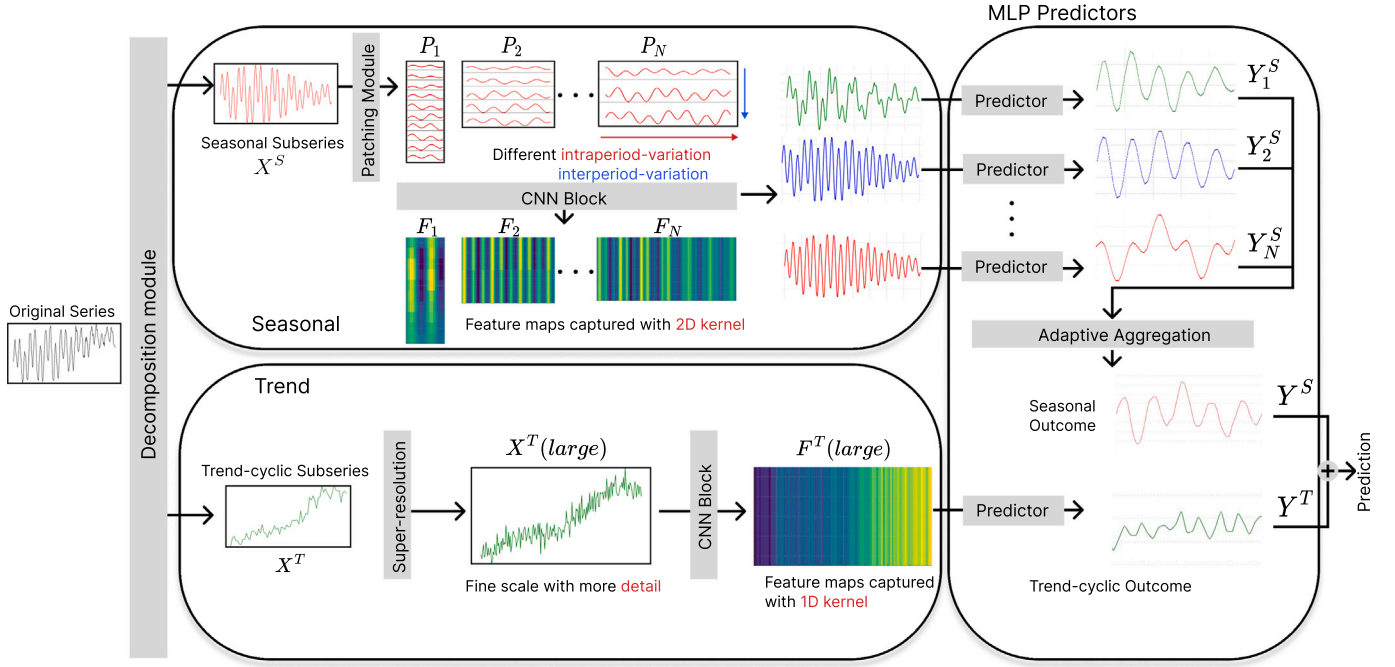
- (i) **Decomposition** (Section 3.1) produces seasonal and trend components  $\mathbf{X}_S, \mathbf{X}_T \in \mathbb{R}^{L \times C}$ .
- (ii) **Seasonal branch** (Section 3.2) unfolds  $\mathbf{X}_S$  into a set of multi-scale 2D patches  $\{\mathbf{P}_i\}_{i=1}^N$ , where  $\mathbf{P}_i \in \mathbb{R}^{(L-w_i+1) \times (w_i C)}$ , and outputs  $\mathbf{Y}_S \in \mathbb{R}^{H \times C}$  through 2D CNN feature extraction and an MLP predictor/aggregation head.
- (iii) **Trend branch** (Sections 3.3 and 3.4) refines  $\mathbf{X}_T$  via trend super-resolution to obtain  $\mathbf{X}_T^{large} \in \mathbb{R}^{(rL) \times C}$  (with SR factor  $r$ ), then applies a lightweight 1D-CNN and an MLP predictor to output  $\mathbf{Y}_T \in \mathbb{R}^{H \times C}$ .
- (iv) **Recomposition** sums the two branch predictions as  $\mathbf{Y} = \mathbf{Y}_S + \mathbf{Y}_T$  (Eq. 14).

Throughout this paper,  $\mathbf{Y} \in \mathbb{R}^{H \times C}$  denotes the *model prediction*, while  $\mathbf{Y}^{gt} \in \mathbb{R}^{H \times C}$  denotes the *ground truth*. Subscripts  $S$  and  $T$  indicate the seasonal and trend branches, respectively (i.e.,  $\mathbf{Y}_S$  and  $\mathbf{Y}_T$ ), and the final recomposed forecast is  $\mathbf{Y} = \mathbf{Y}_S + \mathbf{Y}_T$ .



**Fig. 1.** A comparative analysis of our model's performance against other models. The vertical axis illustrates the average performance across multiple datasets, while the horizontal axis represents the average inference time per run with a batch size of 32. For clarity, the coordinates have been scaled to enhance the distinction between models.





**Fig. 2.** The overall model structure consists of three main components. The original sequence is first decomposed into seasonal and trend components, which are then processed separately to extract relevant features. Finally, the extracted features are input into the MLP predictor for the final prediction. The results are merged to generate the final forecast.

### 3.2. Decomposition module

In real-world scenarios, many time series inherently exhibit both seasonal and trend components [48]. For instance, municipal electricity consumption, commercial vehicle sales, and even the sales of anti-diabetic medications all display a recurring seasonal pattern superimposed on a persistent long-term trend.

The decomposability of time series holds immense practical significance. Through decomposition, more interpretable information can be distilled, such as a seasonal component characterised by recurrence and a trend component marked by continuity. Once decomposed, it becomes evident that most observed fluctuations are systematic and thus readily explained by the data. Moreover, different characteristic components necessitate distinct enhancement strategies: for example, recurring components may be downsampled to capture a richer variety of features, whereas persistent components benefit from upsampling to more effectively reveal their evolving trends. Therefore, in our proposed framework, we likewise employ a decomposition-based approach to analyse the input time series.

As shown in Fig. 2, the original sequence first passes through a decomposition module, which splits the time series into trend and seasonal components, processed separately as Seasonal and Trend branches. This decomposition allows the model to focus more on different temporal scales, resulting in more accurate predictions of long-term dependencies and cyclical variations.

We then use a moving average to smooth the input sequence  $X$  to obtain the trend component  $X^T$ , as:

$$X^T = \frac{1}{w} \sum_{i=t-w/2}^{t+w/2} X_i. \quad (1)$$

Here,  $w$  is the size of the sliding window, and  $X_i$  represents the  $i$ th value in the sequence. To preserve the sequence length, the moving average uses symmetric boundary padding of size  $\lfloor w/2 \rfloor$  on both ends, so that the trend output  $X_T$  remains in  $\mathbb{R}^{L \times C}$  and aligns with  $X$  when computing  $X_S = X - X_T$ .

Then, by subtracting the trend component from the original time series, the seasonal component  $X^S$  is derived as:

$$X^S = X - X^T. \quad (2)$$

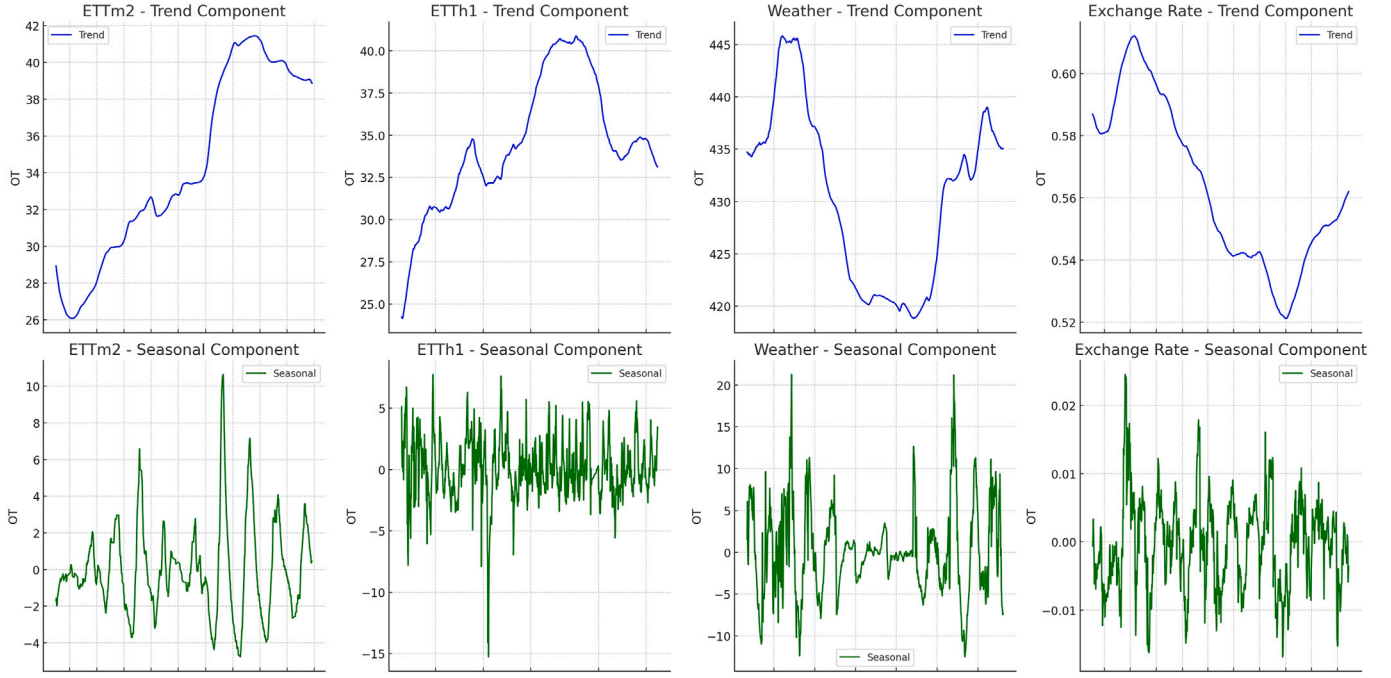
Based on observations, each dataset exhibits different characteristics in terms of seasonality and trend, making it especially important to handle the two branches separately. As shown in Fig. 3,

- **ETtm2:** Strong seasonal characteristics with distinct cyclical patterns; moderate trend strength.
- **ETTh1:** Seasonal characteristics are present, but the trend is relatively stronger.
- **Weather:** Significant seasonal characteristics, with trends influenced by weather fluctuations.
- **Exchange Rate:** Weak seasonal characteristics, but notable trend variations influenced by market dynamics.

Note that the Exchange Rate dataset is used only for qualitative illustration of the decomposition effect in Fig. 3, and is not included in the quantitative benchmark comparison in Section 4.1 because the main comparison table is compiled from published results, most of which do not include this benchmark. To enable the model to better handle the aforementioned scenarios, we apply the following strategies to the seasonal branch and trend branch separately.

### 3.3. Seasonal branch

The Seasonal Branch in this work consists primarily of two key components: a Patching Module and a CNN Block. The Patching Module is responsible for performing multi-scale operations on the seasonal component, effectively converting the sequence into multiple patches to capture information at different temporal scales. This is followed by a CNN Block, which is used to extract inter-period variation and intra-period variation features from each patch, allowing the model to effectively capture the cyclical patterns inherent in the seasonal data.



**Fig. 3.** Comparative decomposition of trend and seasonal components across multiple datasets. From left to right: ETTm2, ETTh1, Weather, and Exchange Rate. (PS. Exchange Rate is included here solely to illustrate decomposition behaviour under a weak-seasonality/strong-trend regime and is not part of the benchmark experiments in Section 4.1).

The concepts of inter-period variation and intra-period variation, introduced in TimesNet [11], refer to two types of variability within time series data. As shown in Fig. 2,

- **Inter-period variation**, as indicated by the blue top-down arrows, represents the variability pattern between different periods in the time series.
- **Intra-period variation**, as indicated by the red arrows pointing from left to right, represents the variability pattern within a single period of the patch block.

The **Patching Module** selects  $N$  different patch sizes to convert the one-dimensional sequence into a two-dimensional sequence. Specifically, the original sequence is segmented according to different frequencies and then stacked together to form a two-dimensional representation, as:

$$X^S \xrightarrow{\text{Patching Module}} \{P_1, P_2, \dots, P_N\}, \quad (3)$$

where  $P_i \in \mathbb{R}^{L_i \times F_i}$  represents the generated  $i$ -th 2D sequence. Here,  $L_i$  is the length (number of rows) and  $F_i$  is the feature dimension (number of columns) of this patch. Formally, let  $X^S \in \mathbb{R}^{L \times C}$  be the seasonal component (length  $L$ ,  $C$  channels). Rather than forming disjoint or strided patches, we employ a sliding-window embedding: we construct each  $P_i$  such that every contiguous window of length  $w_i$  is represented. Specifically,

$$P_i = \mathcal{U}_{w_i}(X^S) \in \mathbb{R}^{(L-w_i+1) \times (w_i C)}, \quad P_i[r, c + jC] = X_{r+c, j}^S, \quad (4)$$

for  $r = 0, \dots, L - w_i$ ,  $c = 0, \dots, w_i - 1$ ,  $j = 0, \dots, C - 1$ . Here, each row of  $P_i$  corresponds to one contiguous subsequence of length  $w_i$ , while each column index  $c$  aligns the same relative phase across subsequences. This “stack-all” unfolding is implemented via a fully overlapping sliding-window embedding: for each start position  $r$ , we extract the window  $X_{r:r+w_i-1}^S$  and include it as a row in  $P_i$ . Because we slide with stride 1 and include every contiguous window, no temporal detail is skipped.

As a result, intra-period variation is captured along the column (phase) dimension, and inter-period variation along the row (cycle) dimension.

The purpose of using patching and multi-scaling here is to enhance the CNN module’s ability to perceive global features. While applying a CNN directly to a one-dimensional sequence allows for the extraction of intraperiod variation—representing short-term temporal features—it struggles to effectively capture interperiod variation, such as seasonal characteristics. After applying patching, the columns of the resulting two-dimensional time series carry seasonal information, which can then be extracted using a 2D CNN kernel. Using multi-scaled patching to capture patterns with different frequencies ensures the extraction of diverse seasonal characteristics. Since the frequencies of seasonal features are not unique, employing multiple patch lengths allows the model to capture varied seasonal patterns effectively.

Specifically, before convolution, each patch  $P_i$  is normalised and reshaped to yield  $\tilde{P}_i \in \mathbb{R}^{L_i \times F_i \times C}$ , which serves as the 2D input feature map for convolution. A 2D kernel  $H \in \mathbb{R}^{u \times v}$  then operates on  $\tilde{P}_i$  as:

$$(H * \tilde{P}_i)[r, c, :] = \sum_{a=0}^{u-1} \sum_{b=0}^{v-1} H[a, b] \tilde{P}_i[r+a, c+b, :], \quad (5)$$

which performs phase-selective filtering along the column (phase) dimension and cycle-wise aggregation along the row (cycle) dimension in a single operation. This formulation illustrates that the combination of patching and 2D convolution jointly models intra-period (phase-wise) and inter-period (cycle-wise) variations within a single operation—something that 1D convolutions on the raw sequence cannot explicitly achieve. Employing multiple window and stride configurations across the  $N$  patching scales further covers unknown or mixed periodicities, forming phase-cycle grids with different temporal resolutions.

The **CNN module** then adopts the Inception Block [49]. Instead of choosing a single kernel size, the Inception module uses multiple convolutional kernels of different sizes (e.g., 1x1, 3x3, 5x5) as well as a max pooling operation, all in parallel within the same layer. The outputs from these different convolutional and pooling layers are concatenated along

the channel dimension, providing a rich set of features from different receptive fields.

This multi-scale approach effectively captures features at various levels of abstraction in the same layer, which is particularly effective for extracting seasonal characteristics, as demonstrated in TimesNet [11]. Depending on specific requirements, other CNN modules can be substituted to improve efficiency or better adapt to the dataset's feature extraction needs.

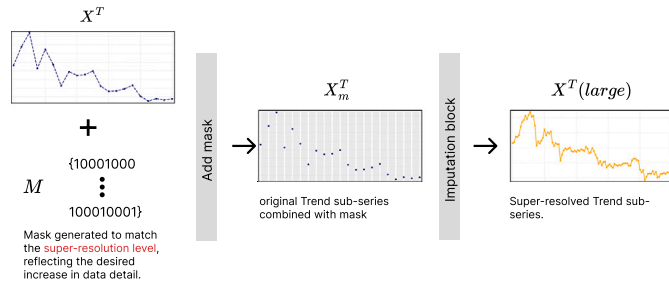
To further illustrate this process, the following formula describes how the Inception Block processes each patch  $P_i$  to extract features at different scales in parallel as:

$$F_i^S = [f_{1 \times 1}(P_i) \parallel f_{3 \times 3}(P_i) \parallel f_{5 \times 5}(P_i) \parallel f_{\text{pool}}(P_i)], \quad i = 1, 2, \dots, N \quad (6)$$

where  $F_i^S$  represents the feature map obtained from patch  $P_i$  after applying the Inception Block,  $f_{1 \times 1}$ ,  $f_{3 \times 3}$  and  $f_{5 \times 5}$  represent convolution operations with kernels of different sizes to capture features at multiple scales, and  $f_{\text{pool}}$  stands for pooling.

### 3.4. Trend branch

Unlike the Seasonal Branch, the Trend Branch does not exhibit strong seasonal patterns. Instead, the Trend component captures the overall directional trend, in which numerous fine-grained fluctuations are also worth learning, as demonstrated in [50,51]. To better capture these fine details, we propose performing super-resolution on the Trend component to acquire more refined details and ultimately enhance the



**Fig. 4.** The super-resolution process of the Trend Branch illustrates the enhancement of temporal resolution for improved trend representation. The binary mask follows a periodic anchor pattern (e.g., 1000 for  $r = 4$ ), where anchors are clamped observations and zeros indicate positions to be imputed.

prediction performance for the Trend. This differentiated approach not only saves computational resources required in the CNN module but also enhances prediction accuracy by focusing on the fine details of the overall directional fluctuations.

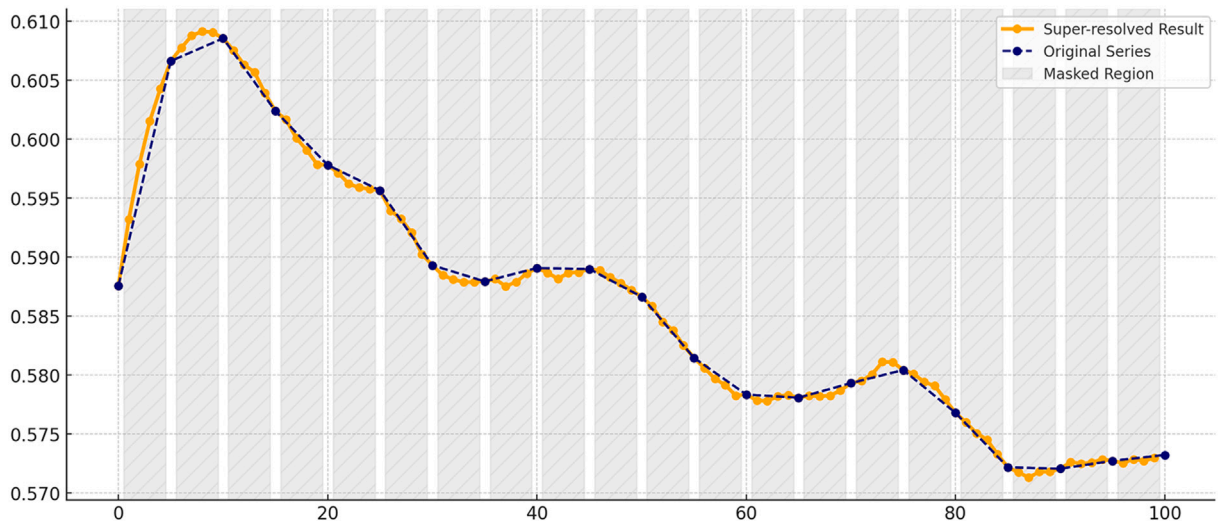
In our work, we adopted the diffusion module from the Structured State Space Diffusion (SSSD) framework [52], a diffusion-based time series imputation model combining diffusion models with structured state space modelling, to carry out our super-resolution process. The SSSD approach uses masking to handle time series interpolation, introducing four types of interpolation masks: RM (random missing), RBM (random block missing), BM (blackout missing), and TF (time series forecasting). In this work, we employ a dedicated **super-resolution mask** with periodic anchors to reflect the upsampling factor  $r$ . Notably, this SSSD-based refinement is applied only to the decomposed trend branch after the moving-average split, and does not modify the 2D seasonal patching pipeline.

As shown in Fig. 4, we consider super-resolution as a special case of conditional interpolation: the coarse trend anchors are treated as observed points on a higher-resolution grid, and the intermediate points are imputed by the diffusion model, yielding a fine-grained trend in Fig. 5. We reuse the SSSD architecture and diffusion schedule, but adapt it to **trend super-resolution** by constructing an SR-specific **periodic-anchor mask** and fine-tuning the model on decomposed trend components under this masking scheme for domain adaptation.

**SR mask construction and anchor selection:** Given a super-resolution factor  $r$  (selected from  $\{2, 4\}$  as in Section 4.4.1), we place the coarse trend samples as *anchors* on a higher-resolution grid and impute the intermediate points via SSSD. Specifically, let  $X_T \in \mathbb{R}^{L \times C}$  be the decomposed trend component. We define the SR grid length as  $L^{SR} = r(L - 1) + 1$  and map each coarse time index  $i \in \{1, \dots, L\}$  to the SR index  $a_i = 1 + (i - 1)r$ . Anchors are therefore *deterministically* chosen as the original trend samples, and the mask  $M \in \{0, 1\}^{L^{SR} \times C}$  is constructed as  $M[a_i, :] = 1$  for all  $i$  and  $M[j, :] = 0$  otherwise. This yields a periodic pattern (e.g., 1000 repeated when  $r = 4$ ), matching the illustration in Fig. 4. The masked input to SSSD is obtained by clamping anchor values and leaving non-anchor positions to be imputed:  $X_m^T = X_T^{SR} \odot M$ , where  $X_T^{SR}$  denotes the trend values placed on the SR grid with anchors at  $\{a_i\}$ .

The specific operation is defined as follows. Let  $X^T$  be the original trend component. We use a mask  $M$ , composed of values 0 and 1, where 1 represents the regions with observed data and 0 represents the regions to be imputed. Thus, the masked trend components are defined as:

$$X_m^T = X^T \cdot M, \quad (7)$$



**Fig. 5.** Comparison of the original trend sub-series with the fine-scaled Trend sub-series.

where the product “ $\cdot$ ” is element-wise multiplication for each index  $i$ .

Using the SSSD model, we impute the missing values to obtain an enhanced trend component with finer granularity:

$$X_{\text{large}}^T = \text{SSSD}(X_m^T). \quad (8)$$

Here,  $X_{\text{large}}^T$  represents the enhanced trend component, which includes imputed values for the masked regions, effectively providing a super-resolved version of the original trend.

Following [52], noise is injected only at missing indices, while anchors remain clamped and act as conditioning. With diffusion step  $t$  and Gaussian noise  $\epsilon$ , the denoiser  $\epsilon_\theta$  is trained with a masked Mean Squared Error (MSE), as:

$$\mathcal{L}_{\text{SSSD}} = \mathbb{E}_{t, \epsilon} \left\| (1-M)\epsilon - (1-M)\epsilon_\theta(X^T \odot M + \epsilon \cdot (1-M), t, M) \right\|_2^2. \quad (9)$$

Thus, the objective focuses the denoising error on the SR (imputed) positions only. At inference, iterative reverse diffusion yields  $X_{\text{large}}^T$  that is consistent with the anchors.

Conceptually, we place the coarse trend anchors on a higher-resolution timeline and construct the mask  $M$  that marks anchors ( $= 1$ ) and gaps ( $= 0$ ). We then initialise the missing positions with noise and run reverse diffusion using SSSD conditioned on  $(X^T \odot M, M)$ , keeping the observed entries clamped so that the generated values remain consistent with the anchors. This process yields the super-resolved trend  $X_{\text{large}}^T$ , which is subsequently fed into the trend head for prediction.

The super-resolved trend component contains additional details that offer more features to be captured, which can significantly aid in the forecasting process. As shown in Fig. 5, the super-resolved trend sequence exhibits a richer set of features, providing more nuanced characteristics and trends before fluctuations in the data points occur, allowing the MLP predictor to make better-informed predictions.

In contrast to the seasonal component, which involves capturing inter-period variation, the trend component does not exhibit such seasonal patterns. As a result, the super-resolved trend component does not require reshaping into a 2D tensor. Instead, we apply a lightweight 1D convolution to  $X_{\text{large}}^T$  to extract local variations, and feed the result into an MLP predictor for the trend forecast.

### 3.5. MLP predictors

After processing through their respective branches, the trend and seasonal components produce distinct feature maps, which are subsequently passed to dedicated MLP predictors. The trend component, having undergone no multi-scale processing, generates a single feature map and is therefore directed to a standalone MLP predictor. This ensures efficient handling of the trend component's global characteristics.

Conversely, the seasonal component, processed through multi-scale techniques, yields multiple feature maps, each capturing different temporal seasonal patterns. These feature maps are individually fed into corresponding MLP predictors, resulting in distinct predictions. This approach allows the model to leverage the unique temporal properties encapsulated at different scales, enhancing its predictive capability for seasonal variations. The trend component prediction is represented as:

$$Y^T = \text{Predictor}_{\text{trend}}(F_{\text{Large}}^T). \quad (10)$$

where  $Y^T$  represents the output prediction for the trend component, and  $F_{\text{Large}}^T$  is the feature map of the trend component obtained after super-resolution and feature extraction. The  $\text{Predictor}_{\text{trend}}$  is the MLP-based module responsible for predicting the future trend values based on  $F_{\text{Large}}^T$ .

The prediction for each seasonal component feature map is given by:

$$Y_i^S = \text{Predictor}_i(F_{S_i}), \quad \forall i \in \{1, 2, \dots, N\}, \quad (11)$$

where  $Y_i^S$  represents the output prediction for the seasonal feature map  $F_{S_i}$ , obtained from multi-scale decomposition of the seasonal component. The corresponding  $\text{Predictor}_i$  processes each feature map to generate  $Y_i^S$ , for  $i = 1, 2, \dots, N$ .

In our approach, the seasonal prediction results are obtained through an adaptive aggregation mechanism inspired by Auto-Correlation [11]. This method effectively utilises amplitude information to determine the relative importance of different seasonal feature components, thereby enhancing the final prediction.

Given the  $N$  seasonal feature maps  $F_1^S, F_2^S, \dots, F_N^S$ , we first compute the normalised importance scores using the softmax function:

$$\hat{A}_1, \hat{A}_2, \dots, \hat{A}_N = \text{Softmax}(A_1, A_2, \dots, A_N), \quad (12)$$

where  $A_i$  represents the amplitude for each feature map  $F_i^S$ . The final aggregated seasonal prediction  $Y^S$  is then computed as:

$$Y^S = \sum_{i=1}^N \hat{A}_i \times F_i^S. \quad (13)$$

This adaptive aggregation ensures that the most informative seasonal features are effectively utilised, thereby improving the prediction accuracy for the seasonal component  $Y^S$  by prioritising features based on their importance.

In our experiments, we explored two different aggregation methods for the feature maps  $F_1, \dots, F_N$  in the Seasonal branch. The first method involves aggregating the feature maps before prediction, allowing for a single prediction to generate the seasonal prediction outputs. The second method aggregates the results after performing predictions on the  $N$  feature maps, which necessitates the use of multiple predictors. Fig. 6 illustrates the two aggregation methods.

Finally, the seasonal prediction  $Y^S$  and the trend prediction  $Y^T$  are summed to obtain the final prediction  $Y$  as:

$$Y = Y^T + Y^S. \quad (14)$$

For supervised forecasting over a horizon  $H$ , we minimise the mean squared error (MSE):

$$\mathcal{L}_{\text{forecast}} = \frac{1}{H} \sum_{h=1}^H (Y_h - Y_h^{\text{gt}})^2, \quad (15)$$

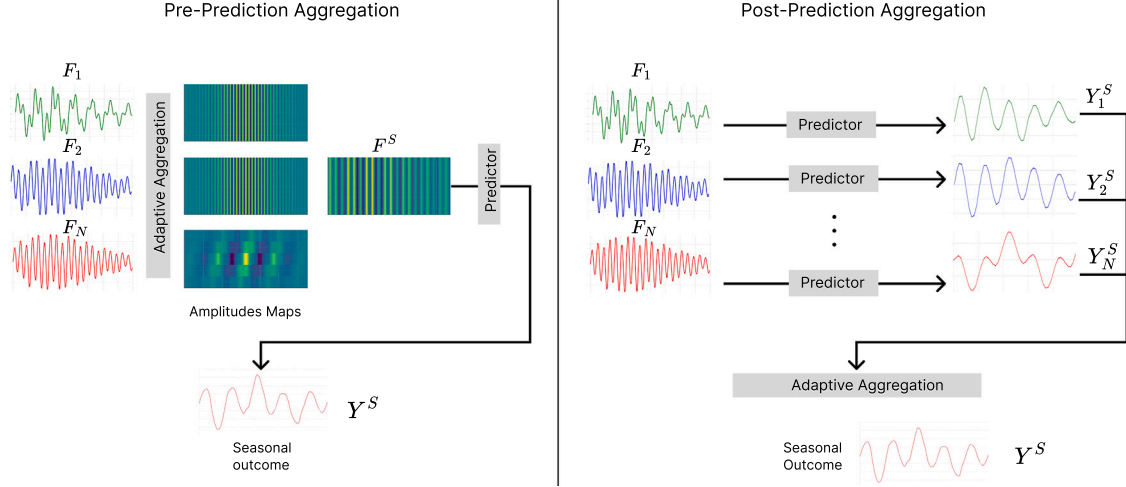
where  $Y_h$  denotes the predicted value at horizon step  $h$  extracted from  $Y$ , and  $Y_h^{\text{gt}}$  denotes the corresponding ground-truth value. Note that Eq. (9) is used only for SSSD pretraining/fine-tuning in the trend super-resolution task, whereas Eq. (15) is used for end-to-end forecasting training.

### 3.6. Forward pass and training dynamics

Given an input window  $\mathbf{X} = \mathbf{Z}_{t-L+1:t} \in \mathbb{R}^{L \times C}$  and forecasting horizon  $H$ , the model produces  $\mathbf{Y} \in \mathbb{R}^{H \times C}$  through the seasonal and trend branches as follows. We first decompose  $\mathbf{X}$  into seasonal  $\mathbf{X}_S$  and trend  $\mathbf{X}_T$  components. The seasonal branch unfolds  $\mathbf{X}^S$  into multi-scale 2D patches  $\{P_i\}$  (see Section 3.3), extracts intra-/inter-period features with CNNs, and produces seasonal predictions  $\{Y_i^S\}$ . These are adaptively aggregated via amplitude-based weights into  $Y^S$ . The trend branch applies SSSD-based super-resolution to  $X^T$  (see Section 3.4), passes the result through a 1D-CNN and MLP, and outputs  $Y^T$ . The final forecast is recomposed as  $Y = Y^S + Y^T$  (Eq. 14).

Training proceeds in two stages. First, the trend SR module is fine-tuned with the masked denoising objective  $\mathcal{L}_{\text{SSSD}}$  (Eq. 9). Then, the full network is trained end-to-end with the forecasting loss  $\mathcal{L}_{\text{forecast}}$  (Eq. 15), using Adam optimiser. Algorithm 1 summarises the forward and training loop.





**Fig. 6.** Comparison of two aggregation methods. Left: pre-prediction aggregation (requires a single predictor) vs. Right: post-prediction aggregation (requires multiple predictors).

**Algorithm 1** Forward pass and training loop (simplified).

- 1: **Input:**  $X \in \mathbb{R}^{L \times C}$ , ground-truth  $Y^{gt} \in \mathbb{R}^{H \times C}$
- 2: **Decomposition:**  $(X^S, X^T) \leftarrow \text{Decompose}(X)$
- 3: **Seasonal branch:**
  - $\{P_i\} \leftarrow \mathcal{U}_{\{2,3,5\}}(X^S)$  ▷ Max-overlap patching
  - $F_i^S \leftarrow \text{CNN}(P_i)$  ▷ 2D conv features
  - $Y_i^S \leftarrow \text{MLP}_i(F_i^S)$  ▷ Patch-level predictions
  - $Y^S \leftarrow \sum_i \hat{A}_i Y_i^S$  ▷ Adaptive aggregation
- 4: **Trend branch:**
  - $X_{\text{large}}^T \leftarrow \text{SSSD}(X^T)$  ▷ SR via diffusion
  - $F^T \leftarrow \text{CNN1D}(X_{\text{large}}^T)$
  - $Y^T \leftarrow \text{MLP}_{\text{trend}}(F^T)$
- 5: **Final recomposition:**  $Y \leftarrow Y^S + Y^T$
- 6: **Training update:** Update parameters  $\theta$  using Adam step on  $\mathcal{L}_{\text{forecast}}$

## 4. Experiments

To assess the effectiveness of the proposed approach, we conduct extensive experiments and benchmark it against state-of-the-art methods, including iTransformer [3], TimeMixer [10], WPMixer [32], TimesNet [11], Crossformer [7], DLinear [23], FEDformer [6], Autoformer [1], and Informer [2].

All the experiments are implemented in PyTorch and conducted on a single NVIDIA GeForce RTX 4070 12GB GPU. We utilise the MSE loss for model training. In the experiments, the number of patches  $P_N$  selected in the seasonal branch and the super-resolution factor used in the fine-scale process of the trend branch are adjusted based on the dataset.

### 4.1. Datasets and experimental protocol

Experiments are conducted on six benchmarks: Weather [1], Traffic, ETTh1, ETTh2, ETTm1, and ETTm2 [2]. All splits are *chronological*. For the four ETT datasets, we adopt the standard long-term time series forecasting (LTSF) partition of 6:2:2 (train/validation/test); for Weather and Traffic, we use 7:1:2. Supervised samples are constructed by a stride-1 sliding window with look-back  $L = 96$  and horizons  $H \in \{96, 192, 336\}$ :

$$(X_{t-L+1:t}, Y_{t+1:t+H}), \quad X_{t-L+1:t} \in \mathbb{R}^{L \times C}, \quad Y_{t+1:t+H} \in \mathbb{R}^{H \times C}.$$

Per-variable z-score normalisation is computed on the *training* split and applied to validation/test; predictions are de-normalised before scoring. All windows are drawn strictly within their respective time partitions to

avoid leakage. For a fair comparison, the main benchmark suite follows the commonly used LTSF protocol and aligns with the datasets reported by the compared baselines. As Exchange Rate is not included in the experimental settings of many representative baseline methods (and thus consistent reported results are unavailable), it is excluded from the quantitative comparisons and used only for decomposition illustration in Fig. 3.

### 4.2. Overall results

As shown in Table 1, our proposed method achieves competitive performance across various benchmarks, and demonstrates strong robustness across datasets with different seasonal–trend characteristics. All baseline results reported in our tables follow the official evaluation protocols. For most baselines, the results are quoted from their original publications. However, WPMixer [32] is an exception: the results reported in the original WPMixer paper are not strictly aligned with our evaluation setting (e.g., data preprocessing and protocol). To ensure a fair and consistent comparison, we have reproduced WPMixer under exactly the same experimental setup as our method (same dataset splits, input length, prediction horizons, and evaluation protocol), as shown in Table 1.

In different scenarios, datasets exhibit varying intensities of seasonal and trend patterns. Previous SOTA models each have their strengths: for example, iTransformer [3] excels at handling stronger trend patterns and shorter prediction windows, such as in the Traffic dataset. On the other hand, WPMixer [32] performs better when predicting long sequence windows with strong seasonal patterns, such as in the ETT dataset. Although WPMixer achieves strong performance on several ETT variants, its accuracy degrades noticeably on *Traffic* across horizons. Since the *Average* row is computed as the arithmetic mean over all evaluated datasets and forecasting horizons (i.e., 6 datasets  $\times$  3 horizons), a large performance drop on a high-error benchmark such as Traffic can outweigh multiple small gains on other datasets, resulting in a lower overall average. This behaviour suggests that Traffic dataset is more challenging and trend-dominant, where accurately modelling long-term drift and irregular changes is critical. Our proposed method achieves a strong balance in the above-mentioned two scenarios. By employing a decomposition approach, the two types of features are enhanced differently. While it may not always surpass the current SOTA models in individual categories, the overall stability of our model ensures its competitiveness across various scenarios. A Wilcoxon signed-rank test across all datasets further confirms that our improvements over baselines are statistically significant ( $p < 0.05$ ). From a practical perspective, the proposed model

**Table 1**

Overall comparison of model performance (MSE ↓) across datasets and window sizes. The best result in each row is **bolded** and the second best is underlined. Baselines: iTransformer [3], TimeMixer [10], WPMixer [32], TimesNet [11], Crossformer [7], DLinear [23], FEDformer [6], Autoformer [1], Informer [2].

| Dataset | Window | Ours         | iTransformer | TimeMixer    | WPMixer      | TimesNet     | Crossformer  | DLinear | FEDformer | Autoformer | Informer |
|---------|--------|--------------|--------------|--------------|--------------|--------------|--------------|---------|-----------|------------|----------|
| Weather | 96     | 0.169        | 0.174        | <b>0.163</b> | <u>0.164</u> | 0.172        | 0.195        | 0.195   | 0.217     | 0.266      | 0.300    |
|         | 192    | 0.215        | 0.221        | <b>0.208</b> | 0.212        | 0.219        | <u>0.209</u> | 0.237   | 0.276     | 0.307      | 0.598    |
|         | 336    | <u>0.249</u> | 0.278        | 0.251        | 0.254        | <b>0.246</b> | <u>0.273</u> | 0.282   | 0.339     | 0.359      | 0.578    |
| Traffic | 96     | <u>0.421</u> | <b>0.395</b> | 0.462        | 0.504        | 0.593        | 0.644        | 0.650   | 0.587     | 0.613      | 0.719    |
|         | 192    | <u>0.439</u> | <b>0.417</b> | 0.473        | 0.518        | 0.617        | 0.665        | 0.598   | 0.604     | 0.616      | 0.696    |
|         | 336    | <u>0.462</u> | <b>0.433</b> | 0.498        | 0.551        | 0.629        | 0.674        | 0.605   | 0.621     | 0.622      | 0.777    |
| ETTh1   | 96     | <b>0.369</b> | 0.386        | <u>0.375</u> | 0.376        | 0.384        | 0.423        | 0.397   | 0.395     | 0.449      | 0.865    |
|         | 192    | <u>0.424</u> | 0.441        | 0.429        | <b>0.415</b> | 0.436        | 0.471        | 0.446   | 0.469     | 0.500      | 1.008    |
|         | 336    | 0.491        | 0.487        | <u>0.484</u> | <b>0.450</b> | 0.638        | 0.570        | 0.489   | 0.530     | 0.521      | 1.107    |
| ETTh2   | 96     | 0.291        | <u>0.288</u> | 0.289        | <b>0.270</b> | 0.340        | 0.745        | 0.340   | 0.358     | 0.346      | 3.755    |
|         | 192    | 0.375        | 0.374        | <u>0.372</u> | <b>0.356</b> | 0.402        | 0.877        | 0.482   | 0.429     | 0.456      | 5.602    |
|         | 336    | 0.391        | 0.415        | <u>0.386</u> | <b>0.355</b> | 0.452        | 1.043        | 0.591   | 0.496     | 0.482      | 4.721    |
| ETTm1   | 96     | <u>0.319</u> | 0.334        | 0.320        | <b>0.302</b> | 0.352        | 0.404        | 0.346   | 0.379     | 0.505      | 0.672    |
|         | 192    | 0.363        | 0.377        | <u>0.361</u> | <b>0.352</b> | 0.390        | 0.450        | 0.382   | 0.426     | 0.553      | 0.795    |
|         | 336    | 0.392        | 0.426        | <u>0.390</u> | <b>0.376</b> | 0.421        | 0.532        | 0.415   | 0.445     | 0.621      | 1.212    |
| ETTm2   | 96     | 0.178        | 0.180        | <u>0.175</u> | <b>0.170</b> | 0.183        | 0.287        | 0.193   | 0.203     | 0.255      | 0.365    |
|         | 192    | 0.243        | 0.250        | <u>0.237</u> | <b>0.227</b> | 0.255        | 0.414        | 0.284   | 0.269     | 0.281      | 0.533    |
|         | 336    | 0.304        | 0.311        | <u>0.298</u> | <b>0.284</b> | 0.309        | 0.597        | 0.382   | 0.325     | 0.339      | 1.363    |
| Average | –      | <b>0.339</b> | 0.344        | 0.343        | <u>0.341</u> | 0.391        | 0.526        | 0.406   | 0.409     | 0.449      | 1.426    |

is designed to provide a favourable accuracy–efficiency trade-off for latency-sensitive forecasting. In many deployment scenarios, timely predictions can be more critical than marginal gains on a single benchmark setting. As shown in Fig. 1 compared with several recent models, our method provides better average performance across the evaluated configurations (see the **Average** row in Table 1), while also offering lower inference latency and a smaller computational footprint under identical settings (Table 6). This makes the proposed framework a practical alternative when both efficiency and stable cross-dataset performance are prioritised.

#### 4.3. Ablation studies

##### 4.3.1. The impact of decomposition and 2D patching

As shown in Table 2, integrating the decomposition method into our Hybrid CNN-MLP framework significantly enhances the model’s performance across multiple benchmark datasets. By separating the seasonal and trend components, the model leverages the CNN branch to capture various repetitive patterns in the seasonal component through 2D convolutional kernels combined with patch-based processing, which enables localised feature extraction and multi-scale perception of periodic dynamics, while employing a super-resolution approach to enhance the trend branch. This design effectively reduces noise interference and mitigates the masking of subtle long-term signals, thereby achieving more accurate and stable predictions. In this setup, the decomposition not only isolates seasonal and trend variations but also structurally activates the 2D seasonal branch, ensuring efficient token compression and improved representation of complex periodic patterns. This is particularly relevant for long-sequence forecasting, such as predictions with a window of 336. The longer the input window, the more features and patterns are present. Consequently, decomposing the sequence and explicitly modelling the seasonal substructure via 2D patching helps to mitigate the noise generated by the mutual interference among various features.

##### 4.3.2. The impact of super-resolving trend signals

In this ablation study, we investigate the impact of super-resolving the trend component. Our approach first decomposes the input series into trend and seasonal components. Unlike the seasonal part—which is characterised by repetitive patterns—the trend component is less

repetitive, and its prediction hinges on capturing its persistent changes and short-term variations. By applying super-resolution to the trend signal, these subtle yet critical features become more pronounced, facilitating a more accurate forecast. As demonstrated in Table 2, methods that incorporate super-resolution consistently yield lower prediction errors across all datasets and forecasting horizons. For instance, on the Weather dataset, the 96-step error is reduced from 0.178 (with decomposition only) to 0.169 when super-resolution is applied. Similar improvements are observed across the Traffic and ETT datasets, underscoring the effectiveness of super-resolving the trend component in enhancing model predictability. Table 2 also provides a direct incremental comparison: the “Without Trend Super-resolution” setting corresponds to the simplest configuration that **skips any trend refinement** (while keeping the same decomposition and backbone). Enabling trend SR therefore quantifies the net benefit of introducing the diffusion-based refinement under an otherwise identical setup. Although Exchange Rate is not included in the main benchmark suite for the quantitative comparison (as discussed above), it is shown in Fig. 3 as a qualitative example to visualise the decomposition behaviour under weak-seasonality/strong-trend dynamics. Within the evaluated benchmarks, ETTh2 has been reported to exhibit *relatively low seasonal variation* (i.e., less pronounced seasonality) compared with other commonly used LTSF datasets [53]. Consistently, Table 2 shows that enabling the trend SR branch yields stable improvements on ETTh2 across horizons, and the gains remain evident at the longest evaluated horizon. This is aligned with the intuition that when seasonal cues are weak, small trend deviations are more likely to accumulate as the forecasting horizon increases. Therefore, the empirical results on ETTh2 and the longest evaluated horizons provide supporting evidence for the expected benefit of the separated trend branch and SR refinement in weak-seasonality and longer-horizon regimes, and a more extensive evaluation along this direction is an important avenue for future work.

##### 4.3.3. Pre- and post aggregation

We first compare the efficacy of the two different aggregation methods for the feature maps  $F_1, \dots, F_N$  in the seasonal branch. The first method aggregates the feature maps before prediction, allowing for a single prediction to generate the seasonal prediction outputs, whereas the second method aggregates the results produced by multiple predictors after performing predictions on the  $N$  feature maps.

**Table 2**

A comparative analysis (MSE ↓) of our model with and without decomposition, as well as with and without the trend signal super-resolution (SR).

| Dataset | Without decomposition          |       |       | With decomposition          |       |       |       |       |       |
|---------|--------------------------------|-------|-------|-----------------------------|-------|-------|-------|-------|-------|
|         | Without Trend Super-resolution |       |       | With Trend Super-resolution |       |       |       |       |       |
|         | 96                             | 192   | 336   | 96                          | 192   | 336   | 96    | 192   | 336   |
| Weather | 0.217                          | 0.286 | 0.342 | 0.178                       | 0.230 | 0.263 | 0.169 | 0.215 | 0.249 |
| Traffic | 0.551                          | 0.577 | 0.608 | 0.447                       | 0.462 | 0.485 | 0.421 | 0.439 | 0.462 |
| ETTh1   | 0.515                          | 0.561 | 0.654 | 0.393                       | 0.453 | 0.522 | 0.369 | 0.424 | 0.491 |
| ETTh2   | 0.354                          | 0.481 | 0.515 | 0.309                       | 0.396 | 0.418 | 0.291 | 0.375 | 0.391 |
| ETTM1   | 0.415                          | 0.485 | 0.502 | 0.336                       | 0.387 | 0.419 | 0.319 | 0.363 | 0.392 |
| ETTM2   | 0.217                          | 0.293 | 0.392 | 0.189                       | 0.261 | 0.323 | 0.178 | 0.243 | 0.304 |

**Table 3**

Comparison of the performance (MSE ↓) obtained using the two different aggregation methods with varied prediction windows.

| Dataset | Pre-Prediction aggregation |       |       | Post-Prediction aggregation |       |       |
|---------|----------------------------|-------|-------|-----------------------------|-------|-------|
|         | 96                         | 192   | 336   | 96                          | 192   | 336   |
| Weather | 0.178                      | 0.227 | 0.265 | 0.169                       | 0.215 | 0.249 |
| Traffic | 0.429                      | 0.453 | 0.480 | 0.421                       | 0.439 | 0.462 |
| ETTh1   | 0.397                      | 0.455 | 0.510 | 0.369                       | 0.424 | 0.491 |
| ETTh2   | 0.294                      | 0.383 | 0.402 | 0.291                       | 0.375 | 0.391 |
| ETTM1   | 0.325                      | 0.374 | 0.398 | 0.319                       | 0.363 | 0.392 |
| ETTM2   | 0.181                      | 0.257 | 0.335 | 0.178                       | 0.243 | 0.304 |

As shown in Table 3, the Post-aggregation method, which employs multiple predictors, achieves better performance. However, the Pre-aggregation method also delivers acceptable results, with an average difference of 3.91%. Considering factors such as computational efficiency, the Pre-aggregation method remains a viable option. Nonetheless, for the purpose of comparison, we adopted the Post-aggregation method as the aggregation approach for our model (Fig. 7).

#### 4.3.4. Impact of the patch size

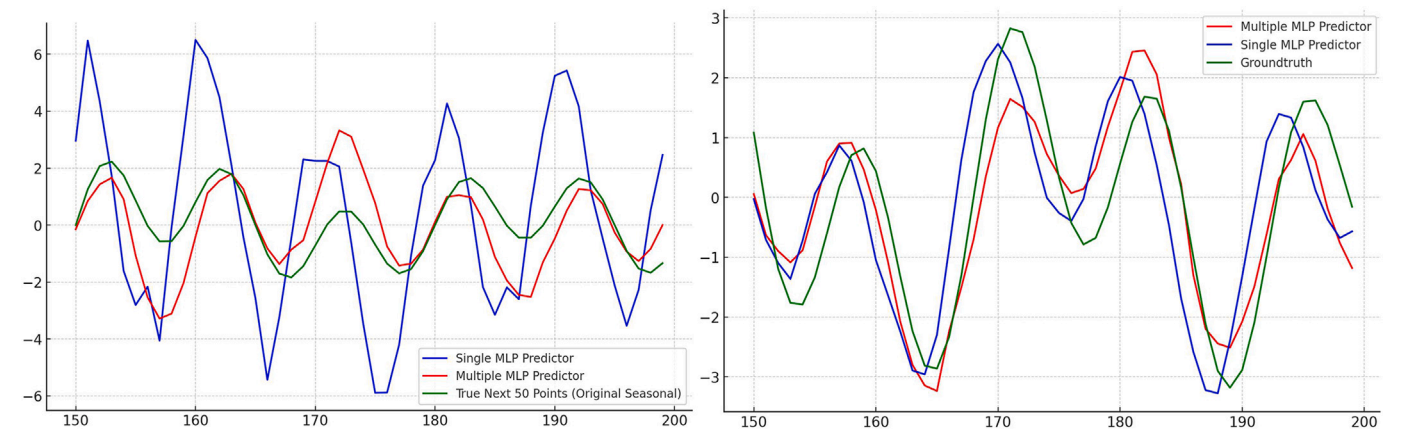
Another set of experiments focuses on comparing different patch sizes. By using various patch sizes, seasonal features at different frequencies can be extracted. The selection of patch sizes can follow approaches from TimesNet [11], where amplitude is utilised as a criterion. However, this approach increases computational costs, and in some cases, low-frequency features may be overlooked.

Naturally, the choice of patch size is also highly dependent on the dataset itself, as different datasets exhibit seasonal features at varying frequencies. As shown in Tables 4 and 5, certain datasets demonstrate

greater sensitivity to specific patch sizes under certain conditions. In our experiments, we observed that treating patch size as a hyperparameter during training and selecting a set of prime numbers as patch sizes can maximise the capture of seasonal features across various frequencies. Conversely, using patch sizes with multiple relationships may lead to feature redundancy, thereby neglecting other frequency bands. To further isolate the effect of multi-scale design, Table 5 reports a single-scale variant that uses only one patch size  $p \in \{2, 3, 5, 11\}$  under the same evaluation protocol. Compared with the multi-scale settings in Table 4, the single-scale results are consistently worse across datasets and horizons, suggesting that multi-scale patching better captures complementary seasonal frequencies and improves robustness, while also reducing sensitivity to the choice of a single patch length. In practical applications, the decision to use automated patch size selection or the choice of specific values for patch sizes can be adjusted based on factors such as computational resources, dataset composition, and pre-training considerations.

#### 4.4. Computational efficiency analysis

Table 6 reports a detailed comparison of computational efficiency and resource cost across representative Transformer and MLP-based forecasting models under identical hardware and data settings (NVIDIA RTX 4070 12 GB,  $L=96$ ,  $H=96$ , batch size = 32). Importantly, the reported numbers for our model in Table 6 are end-to-end measurements of the full pipeline, including the trend SR module, which makes the accuracy gain in Table 2 and the associated resource overhead transparent for practitioners. Transformer variants such as Autoformer [1], TimesNet [11], and iTransformer [3] exhibit the highest computational demand due to quadratic attention and multi-layer projection, leading to larger latency and memory footprints. In contrast, lightweight architectures such as DLinear [23], TimeMixer [10], and WPMixer [32]



**Fig. 7.** Comparison of prediction results between pre-aggregation (blue) and post-aggregation (red) methods. Two segments are selected from the ETT dataset, and fluctuation ranges differ between the two cases, with post-aggregation demonstrating better prediction performance in both cases.

**Table 4**

Performance (MSE ↓) comparison with different patch sizes (2 4 8, 3 5 11, 3 9 12, and 2 3 5). The bottom row shows the overall average for each patch size group.

| Patch size  | 2     | 4     | 8     | 3     | 5     | 11    | 3     | 9     | 12    | 2     | 3     | 5     |
|-------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| Window size | 96    | 192   | 336   | 96    | 192   | 336   | 96    | 192   | 336   | 96    | 192   | 336   |
| Weather     | 0.181 | 0.238 | 0.277 | 0.169 | 0.215 | 0.249 | 0.176 | 0.224 | 0.263 | 0.171 | 0.220 | 0.250 |
| Traffic     | 0.438 | 0.471 | 0.492 | 0.421 | 0.439 | 0.462 | 0.442 | 0.455 | 0.478 | 0.426 | 0.439 | 0.468 |
| ETTh1       | 0.365 | 0.421 | 0.485 | 0.369 | 0.424 | 0.491 | 0.378 | 0.447 | 0.513 | 0.373 | 0.429 | 0.493 |
| ETTh2       | 0.304 | 0.392 | 0.407 | 0.291 | 0.375 | 0.391 | 0.297 | 0.386 | 0.413 | 0.298 | 0.376 | 0.397 |
| ETTm1       | 0.328 | 0.381 | 0.410 | 0.319 | 0.363 | 0.392 | 0.327 | 0.377 | 0.406 | 0.319 | 0.369 | 0.391 |
| ETTm2       | 0.186 | 0.262 | 0.347 | 0.178 | 0.243 | 0.304 | 0.181 | 0.246 | 0.312 | 0.179 | 0.245 | 0.308 |
| Average     | 0.300 | 0.361 | 0.403 | 0.291 | 0.343 | 0.382 | 0.300 | 0.356 | 0.398 | 0.294 | 0.346 | 0.384 |

**Table 5**

Single-scale ablation: Performance (MSE ↓) comparison with single patch size  $p \in \{2, 3, 5, 11\}$ . The bottom row shows the overall average for each patch size.

| Patch size | 2     |       |       | 3     |       |       | 5     |       |       | 11    |       |       |
|------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| Horizon    | 96    | 192   | 336   | 96    | 192   | 336   | 96    | 192   | 336   | 96    | 192   | 336   |
| Weather    | 0.196 | 0.258 | 0.300 | 0.187 | 0.244 | 0.284 | 0.195 | 0.253 | 0.289 | 0.204 | 0.263 | 0.301 |
| Traffic    | 0.471 | 0.507 | 0.530 | 0.462 | 0.485 | 0.506 | 0.465 | 0.489 | 0.514 | 0.478 | 0.498 | 0.521 |
| ETTh1      | 0.396 | 0.459 | 0.524 | 0.398 | 0.462 | 0.530 | 0.401 | 0.471 | 0.546 | 0.411 | 0.485 | 0.556 |
| ETTh2      | 0.330 | 0.423 | 0.439 | 0.316 | 0.409 | 0.428 | 0.322 | 0.416 | 0.441 | 0.335 | 0.425 | 0.446 |
| ETTm1      | 0.353 | 0.412 | 0.444 | 0.345 | 0.396 | 0.424 | 0.351 | 0.406 | 0.435 | 0.358 | 0.414 | 0.444 |
| ETTm2      | 0.201 | 0.284 | 0.376 | 0.193 | 0.268 | 0.332 | 0.197 | 0.273 | 0.340 | 0.205 | 0.278 | 0.348 |
| Average    | 0.325 | 0.391 | 0.436 | 0.317 | 0.377 | 0.417 | 0.322 | 0.385 | 0.428 | 0.332 | 0.394 | 0.436 |

**Table 6**

Computational efficiency and resource cost comparison under identical settings (NVIDIA RTX 4070 12 GB,  $L = 96$ ,  $H = 96$ , batch size = 32). Transformer-based models (e.g., Autoformer [1], TimesNet [11], iTransformer [3]) employ high-dimensional embeddings ( $d_{\text{model}} \approx 512$ ), multi-head attention ( $n_{\text{heads}} = 8$ ), and multiple encoder layers ( $n_{\text{layers}} = 2$ ).

| Model                      | #Params (M)↓ | FLOPs (G)↓  | Latency (ms/batch)↓ | Peak VRAM (GB)↓ |
|----------------------------|--------------|-------------|---------------------|-----------------|
| Autoformer [1]             | 26.4         | 34.7        | 16.2                | 7.4             |
| TimesNet [11]              | 22.1         | 28.9        | 14.7                | 6.9             |
| iTransformer [3]           | 19.5         | 25.3        | 13.5                | 6.2             |
| WPMixer [32]               | 0.8          | 0.23        | 6.8                 | 4.8             |
| DLinear [23]               | 0.14         | 0.23        | 3.5                 | 2.1             |
| TimeMixer [10]             | 0.9          | 2.77        | 7.2                 | 4.5             |
| Ours (2D Patch + Trend SR) | <b>0.95</b>  | <b>0.58</b> | <b>5.4</b>          | <b>3.6</b>      |

substantially reduce computational complexity through simplified mixing mechanisms.

Although our method presents moderately higher FLOPs than WPMixer [32] (owing to the inclusion of the 2D convolutional branch and trend super-resolution modules), it achieves lower inference latency and peak VRAM consumption thanks to its architectural efficiency: the 2D seasonal patching module substantially compresses the token density before convolutional processing, enabling highly parallelised GPU computation with minimal kernel-launch overhead. Moreover, the trend super-resolution branch performs localised reconstruction only on coarse trend anchors, generating short-lived intermediate activations without expanding the full feature map. Consequently, the actual wall-clock runtime and memory footprint are reduced, even under slightly higher arithmetic complexity.

In contrast, WPMixer's multi-resolution wavelet decomposition and frequency-domain mixing, while lightweight in FLOPs, introduce multiple small-scale transformation kernels and intermediate buffers that increase scheduling overhead and activation storage. This structural difference explains why our hybrid CNN-MLP architecture demonstrates higher GPU utilisation efficiency and lower runtime cost, despite marginally greater computational volume. Overall, the proposed model achieves a balanced trade-off between computational efficiency, forecasting accuracy, and architectural interpretability, maintaining state-of-the-art efficiency among non-transformer-based approaches.

#### 4.4.1. Hyperparameter tuning protocol

We tune hyperparameters on the validation split via *random search* over predefined ranges and select the configuration with the lowest validation MSE. The search space includes: learning rate  $\in [1 \times 10^{-5}, 1 \times 10^{-3}]$  (log-uniform); batch size  $\in \{16, 32, 64\}$ ; hidden dimension  $\in \{128, 256, 512\}$ ; dropout  $\in [0.0, 0.3]$ ; MLP depth  $\in \{2, 3\}$ ; Inception channels per branch  $\in \{32, 64\}$ ; and the **seasonal multi-scale patch set** fixed to  $\{2, 3, 5\}$  with the max-overlap stacking described in Section 3.2. For the trend branch, the SSSD super-resolution (SR) factor is chosen from  $\{2, 4\}$  to match dataset granularity. Unless stated, the optimiser is Adam with weight decay  $1 \times 10^{-4}$ .

Table 7 summarises the selected configurations for three representative datasets (the same protocol is used on the remaining benchmarks). In all cases, the seasonal patch set  $\{2, 3, 5\}$  is adopted as it yielded the best validation performance as shown in Table 4.

**Table 7**

Selected hyperparameters from validation-based random search.

| Dataset | LR                 | Batch | Hidden | Dropout | $\{w_i\}$ (seasonal) | SR factor |
|---------|--------------------|-------|--------|---------|----------------------|-----------|
| Weather | $5 \times 10^{-4}$ | 32    | 256    | 0.1     | $\{2, 3, 5\}$        | 2         |
| ETTh1   | $5 \times 10^{-4}$ | 32    | 256    | 0.1     | $\{2, 3, 5\}$        | 4         |
| Traffic | $3 \times 10^{-4}$ | 64    | 256    | 0.1     | $\{2, 3, 5\}$        | 2         |



## 5. Conclusion

We have proposed a hybrid CNN-MLP framework that leverages seasonal-trend decomposition, multi-scale patching, and super-resolution to address the limitations of purely self-attention- or MLP-based models in time series forecasting. By separating the recurring seasonal component from the persistent trend component, the proposed method effectively captures both global and local patterns. Experimental results on six public benchmarks demonstrate that the proposed model achieves competitive performance with a favourable accuracy–efficiency trade-off compared with representative baselines (e.g., TimeMixer and iTransformer). In particular, the average results summarised in Table 1 highlight the stability of the proposed approach across diverse forecasting scenarios, while Table 5 further confirms its efficiency advantages under identical settings.

Moreover, ablation studies highlight the benefits of decomposition and super-resolution in enhancing predictive accuracy, especially for long-sequence forecasting. Taken together, these findings confirm the robustness and practicality of the proposed approach across diverse forecasting scenarios.

In future work, we plan to extend this framework toward spatio-temporal forecasting tasks, where the proposed decomposition and super-resolution modules could enhance spatio-temporal consistency and support online model adaptation under non-stationary conditions. Meanwhile, current limitations include the reliance on predefined seasonal windows, which could be alleviated by adaptive patching or self-tuning decomposition mechanisms in future research. The datasets in our experiments are of high quality and often preprocessed to fill missing values or maintain regular sampling. However, in real-world deployment, we may face irregular sampling, missing or corrupted measurements, and variable reporting intervals, which can degrade forecast accuracy. Prior works have studied learning from irregularly sampled time series [54] and robust interpolation strategies [55]. In future work, our super-resolution module could be adapted to impute or refine missing segments by inferring fine-grained trends, potentially mitigating data irregularities.

## CRedit authorship contribution statement

**Yuwei Chen:** Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Wenjing Jia:** Writing – review & editing, Supervision, Methodology, Investigation, Formal analysis, Conceptualization. **Qiang Wu:** Writing – review & editing, Supervision, Methodology, Investigation, Formal analysis, Conceptualization.

## Ethics approval

This article does not contain studies involving human participants or animals.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

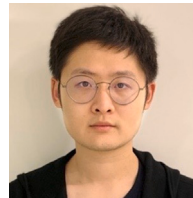
Data will be made available on request.

## References

- [1] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: decomposition transformers with auto-correlation for long-term series forecasting, *NeurIPS* 34 (2021) 22419–22430.
- [2] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, et al., Informer: beyond efficient transformer for long sequence time-series forecasting, in: *AAAI*, vol. 35, 2021, pp. 11106–11115.
- [3] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, et al., ITransformer: inverted transformers are effective for time series forecasting, in: *ICLP*, 2023.
- [4] M.-H. Guo, Z.-N. Liu, T.-J. Mu, S.-M. Hu, Beyond self-attention: external attention using two linear layers for visual tasks, *Trans. PAMI* 45 (5) (2022) 5436–5447.
- [5] B. Peng, Y. Ding, W. Kang, Metaformer: a transformer that tends to mine metaphysical-level information, *Sensors* 23 (11) (2023) 5093.
- [6] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, Fedformer: frequency enhanced decomposed transformer for long-term series forecasting, in: *ICML, PMLR*, 2022, pp. 27268–27286.
- [7] Y. Zhang, J. Yan, Crossformer: transformer utilizing cross-dimension dependency for multivariate time series forecasting, in: *ICLR*, 2022.
- [8] Y. Zhao, G. Wang, C. Tang, C. Luo, W. Zeng, Z.-J. Zha, A battle of network structures: An empirical study of CNN, transformer, and MLP, *arXiv preprint arXiv:2108.13002*, 2021.
- [9] C. Tang, Y. Zhao, G. Wang, C. Luo, W. Xie, W. Zeng, Sparse MLP for image recognition: is self-attention really necessary? in: *AAAI*, vol. 36, 2022, pp. 2344–2351.
- [10] S. Wang, H. Wu, X. Shi, T. Hu, H. Luo, L. Ma, et al., Timemixer: decomposable multiscale mixing for time series forecasting, in: *International Conference on Learning Representations (ICLR)*, 2024.
- [11] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, TimesNet: temporal 2d-variation modeling for general time series analysis, in: *International Conference on Learning Representations (ICLR)*, 2023.
- [12] H. Hewamalage, C. Bergmeir, K. Bandara, Recurrent neural networks for time series forecasting: current status and future directions, *Int. J. Forecast.* 37 (1) (2021) 388–427.
- [13] S. Siami-Namini, N. Tavakoli, A.S. Namin, A comparison of ARIMA and LSTM in forecasting time series, in: *2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA)*, IEEE, 2018, pp. 1394–1401.
- [14] J. Wang, Q. Gu, J. Wu, G. Liu, Z. Xiong, Traffic speed prediction and congestion source exploration: a deep learning method, in: *ICDM, IEEE*, 2016, pp. 499–508.
- [15] S. Li, W. Li, C. Cook, C. Zhu, Y. Gao, Independently recurrent neural network (indrn): building a longer and deeper RNN, in: *CVPR*, 2018, pp. 5457–5466.
- [16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, et al., Attention is all you need, *NeurIPS* 30 (2017).
- [17] D. Luo, X. Wang, Deformableletst: transformer for time series forecasting without over-reliance on patching, *Adv. Neural Inf. Process. Syst.* 37 (2024) 88003–88044.
- [18] S. Wang, B.Z. Li, M. Khabsa, H. Fang, H. Ma, Linformer: Self-attention with linear complexity, *arXiv preprint arXiv:2006.04768*, 2020.
- [19] I. Beltagy, M.E. Peters, A. Cohan, Longformer: The long-document transformer, *arXiv preprint arXiv:2004.05150*, 2020.
- [20] D. Du, B. Su, Z. Wei, Preformer: predictive transformer with multi-scale segment-wise correlations for long-term time series forecasting, in: *ICASSP, IEEE*, 2023, pp. 1–5.
- [21] C. Yang, Y. Wang, B. Yang, J. Chen, Graformer: a gated residual attention transformer for multivariate time series forecasting, *Neurocomputing* 581 (2024) 127466.
- [22] X. Zhang, J. Wang, Y. Bai, L. Zhang, Y. Lin, Tf4tf: multi-semantic modeling within the time-frequency domain for long-term time-series forecasting, *Neurocomputing* 617 (2025) 128913.
- [23] A. Zeng, M. Chen, L. Zhang, Q. Xu, Are transformers effective for time series forecasting? in: *AAAI*, vol. 37, 2023, pp. 11121–11128.
- [24] Y. Hu, P. Liu, P. Zhu, D. Cheng, T. Dai, Adaptive multi-scale decomposition framework for time series forecasting, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025, pp. 17359–17367.
- [25] Y. Cao, Z. Tian, W. Guo, X. Liu, Mspatch: a multi-scale patch mixing framework for multivariate time series forecasting, *Expert Syst. Appl.* 273 (2025) 126849, <https://doi.org/10.1016/j.eswa.2025.126849>
- [26] D. Oikonomou, L. Leontaris, N. Dimitriou, D. Tzovaras, Time-series forecasting in industrial environments: a performance study and a novel late fusion framework, *IEEE Sens. J.* 25 (4) (2025) 7681–7697, <https://doi.org/10.1109/JSEN.2025.3526362>
- [27] I.R. Widiasari, L.E. Nugroho, et al., Deep learning multilayer perceptron (MLP) for flood prediction model using wireless sensor network based hydrology time series data mining, in: *2017 International Conference on Innovative and Creative Information Technology (ICITech)*, IEEE, 2017, pp. 1–5.
- [28] R. Ebrahimpour, H. Nikoo, S. Masoudnia, M.R. Yousefi, M.S. Ghaemi, Mixture of mlp-experts for trend forecasting of time series: a case study of the Tehran stock exchange, *Int. J. Forecast.* 27 (3) (2011) 804–816.
- [29] H. Wang, Y. Mo, H. Dai, N. Yin, S. Fan, B. Li, Ftmlp: MLP with feature-temporal block for multivariate time series forecasting, *Neurocomputing* 607 (2024) 128365.
- [30] H. Jiang, D. Liu, X. Ding, Y. Chen, H. Li, TCM: an efficient lightweight mlp-based network with affine transformation for long-term time series forecasting, *Neurocomputing* 617 (2025) 128960.
- [31] K. Yi, Q. Zhang, W. Fan, S. Wang, P. Wang, H. He, et al., Frequency-domain MLPs are more effective learners in time series forecasting, *NeurIPS* 36 (2024).
- [32] M.M.N. Murad, M. Aktukmak, Y. Yilmaz, Wpmixer: efficient multi-resolution mixing for long-term time series forecasting, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025, pp. 19581–19588.
- [33] M. Kirisci, O. Cagcag Yolcu, A new Cnn-based model for financial time series: Taix and Ftse stocks forecasting, *Neural Process. Lett.* 54 (4) (2022) 3357–3374.
- [34] H. Cheng, Z. Xie, Y. Shi, N. Xiong, Multi-step data prediction in wireless sensor networks based on one-dimensional CNN and bidirectional LSTM, *IEEE Access* 7 (2019) 117883–117896.

- [35] N. Ranjan, S. Bhandari, H.P. Zhao, H. Kim, P. Khan, City-wide traffic congestion prediction based on CNN, LSTM and transpose CNN, *IEEE Access* 8 (2020) 81606–81620.
- [36] R. Nematirad, A. Pahwa, B. Natarajan, Times2D: multi-period decomposition and derivative mapping for general time series forecasting, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025, pp. 19651–19658.
- [37] Y. Nie, N.H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: long-term forecasting with transformers, in: *International Conference on Learning Representations (ICLR)*, 2023.
- [38] Q. Huang, L. Shen, R. Zhang, J. Cheng, S. Ding, Z. Zhou, et al., Hdmixer: hierarchical dependency with extendable patch for multivariate time series forecasting, in: *AAAI*, vol. 38, 2024, pp. 12608–12616.
- [39] K. Han, Y. Wang, H. Chen, X. Chen, J. Guo, Z. Liu, et al., A survey on vision Transformer, *TPAMI* 45 (1) (2022) 87–110.
- [40] W. Wang, E. Xie, X. Li, D.-P. Fan, K. Song, D. Liang, et al., Pyramid vision transformer: a versatile backbone for dense prediction without convolutions, in: *ICCV*, 2021, pp. 568–578.
- [41] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, et al., Swin transformer: hierarchical vision transformer using shifted windows, in: *ICCV*, 2021, pp. 10012–10022.
- [42] Z. Shao, Z. Zhang, F. Wang, Y. Xu, Pre-training enhanced spatial-temporal graph neural network for multivariate time series forecasting, in: *ACM SIGKDD*, 2022, pp. 1567–1577.
- [43] Y. Nie, N.H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: long-term forecasting with transformers, in: *ICLP*, 2022.
- [44] H. Wang, B. Li, S. Fan, Y. Wu, X. Liu, Timesql: improving multivariate time series forecasting with multi-scale patching and smooth quadratic loss, *Information Sci.* 671 (2024) 120652.
- [45] P. Chen, Y. Zhang, Y. Cheng, Y. Shu, Y. Wang, Q. Wen, et al., Pathformer: multi-scale transformers with adaptive pathways for time series forecasting, in: *International Conference on Learning Representations (ICLR)*, 2024.
- [46] S. Zhong, S. Song, W. Zhuo, G. Li, Y. Liu, S.-H.G. Chan, A multi-scale decomposition mlp-mixer for time series analysis, *Proc. VLDB Endow.* 17 (7) (2024) 1723–1736.
- [47] Y. Zhang, L. Ma, S. Pal, Y. Zhang, M. Coates, Multi-resolution time-series transformer for long-term forecasting, in: *International Conference on Artificial Intelligence and Statistics*, PMLR, 2024, pp. 4222–4230.
- [48] K. Bandara, R.J. Hyndman, C. Bergmeir, Mstl: a seasonal-trend decomposition algorithm for time series with multiple seasonal patterns, *Int. J. Oper. Res.* 52 (1) (2025) 79–98.
- [49] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, et al., Going deeper with convolutions, in: *CVPR*, 2015, pp. 1–9.
- [50] H. Wang, J. Peng, F. Huang, J. Wang, J. Chen, Y. Xiao, Micn: multi-scale local and global context modeling for long-term series forecasting, in: *ICLR*, 2023.
- [51] Y. Chen, W. Jia, Q. Wu, Fine-scale deep learning model for time series forecasting, *Appl. Intell.* 54 (20) (2024) 10072–10083.
- [52] J.M. Lopez Alcaraz, N. Strodthoff, Diffusion-based time series imputation and forecasting with structured state space models, *Trans. Mach. Learn. Res.* (2023) <https://openreview.net/forum?id=hHilbk7ApW>.
- [53] W. Ye, S. Deng, Q. Zou, N. Gui, Frequency adaptive normalization for non-stationary time series forecasting, in: *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024.
- [54] S.C.-X. Li, B. Marlin, Learning from irregularly-sampled time series: a missing data perspective, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 5937–5946.
- [55] A. Cini, I. Marisca, C. Alippi, Filling the g.ap.s: multivariate time series imputation by graph neural networks, in: *International Conference on Learning Representations*, 2022, <https://openreview.net/forum?id=kOu3-S3wJ7>.

## Author biography



**Yuwei Chen** received the BSc degree in Software Engineering from the University of New South Wales (UNSW) in 2016. He obtained his Master's degree in Information Technology from the University of Technology Sydney (UTS) in 2021. He is currently pursuing the Ph.D. degree with the Faculty of Engineering and Information Technology (FEIT) at UTS. His main research interests include time series forecasting and deep learning.



**Qiang Wu** received the BEng and Meng degrees in electronic engineering from the Harbin Institute of Technology, Harbin, China, in 1996 and 1998, respectively, and the Ph.D. degree in computing science from the University of Technology Sydney, Sydney, Australia, in 2004. He is currently a Professor and a Core Member of the Global Big Data Technologies Centre, University of Technology Sydney. He has published more than 70 refereed papers, including those published in prestigious journals and top international conferences. His major research interests include computer vision, image processing, pattern recognition, machine learning and multimedia processing. He has served as the chair and/or a Programme Committee Member for a number of international conferences.



**Wenjing Jia** received her Ph.D. degree in Computing Sciences from the University of Technology Sydney in 2007. She is currently an Associate Professor at the Faculty of Engineering and Information Technology (FEIT), University of Technology Sydney (UTS). Her research falls in the fields of image processing and analysis, computer vision, and pattern recognition.